СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....2
1 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ.....4
1.1 Микросервисная архитектура.....4
1.2 Централизованная конфигурация и Git.....4
1.3 API Gateway, Discovery и балансировка.....5
1.4 OAuth2 и межсервисные вызовы.....5
1.5 Раздельное хранение и согласованность.....6
1.6 Связь платформы с ГОСТ Р 56939—2024.....6
2 ПРАКТИЧЕСКАЯ ЧАСТЬ.....7
2.1 Архитектура и средства разработки.....7
2.2 Модель данных.....9
2.3 REST API и валидация запросов.....10
2.4 Авторизация OAuth2 и права доступа.....10
2.5 Конфигурация из Git.....11
2.6 Gateway и обнаружение сервисов.....11
2.7 Взаимодействие через FeignClient.....12
2.8 Раздельные базы данных и миграции.....13
2.9 Управление состоянием запуска.....13
2.10 Модульное тестирование.....14
2.11 Запуск и сквозная демонстрация.....15
2.12 Проверка балансировки нагрузки.....17
2.13 Пять этапов полного сценария.....18
2.14 Сквозной requestId через несколько сервисов.....22
2.15 Асинхронное событие RabbitMQ.....23
2.16 Отказ одного сервиса и восстановление.....24
2.17 Выполнение требований задания.....25
ЗАКЛЮЧЕНИЕ.....27
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....28
ПРИЛОЖЕНИЕ А.....30
А.1 Структура проекта.....30
А.2 Команды воспроизведения.....31
А.3 Репозиторий исходного кода.....32

ВВЕДЕНИЕ

Подготовка среды для испытаний классического клиент-серверного ПО включает выбор операционных систем, настройку серверной и клиентской частей, зависимостей и сетевого взаимодействия. При ручной подготовке труднее повторить условия проверки и связать результат с конкретной версией программы. Для автоматизации этих действий требуется управляющая платформа, которая учитывает объекты испытаний, параметры стендов и выполненные запуски.

Тема выпускной квалификационной работы — «Инфраструктура-как-сервис: автоматизированная платформа тестирования классического клиент-серверного ПО по ГОСТ Р 56939—2024». В практической работе создаётся её начальная микросервисная реализация. На первом этапе реализуются управляющие API и инфраструктурные компоненты Spring Cloud, которые впоследствии могут быть дополнены провайдером виртуальных машин и исполнителем тестов.

Цель работы — разработать и проверить микросервисный прототип платформы с OAuth2-авторизацией, централизованной конфигурацией из Git, API Gateway, обнаружением сервисов, балансировкой нагрузки и отдельными базами данных предметных сервисов; проверить requestId и обмен событиями RabbitMQ.

Для достижения цели поставлены следующие задачи:

– выделить не менее трёх предметных микросервисов и определить их ответственность;

– реализовать регистрацию версий ПО, шаблонов стендов и тестовых запусков;

– настроить выдачу и проверку OAuth2-токенов с разграничением операций чтения и записи;

– реализовать Config Server с Git в качестве источника конфигурации;

– настроить API Gateway, Service Discovery и балансировщик нагрузки;

– организовать взаимодействие предметных сервисов через FeignClient;

– обеспечить каждому предметному сервису отдельный экземпляр PostgreSQL;

– подготовить unit-тесты, файлы развёртывания и демонстрационный сценарий;

– проверить локальный запуск, обработку ошибок и распределение запросов между репликами;

– добавить сквозной requestId, обмен событиями RabbitMQ и снять протокол отказа сервиса.

Результаты относятся к локальному стенду Docker Compose. Обязательное по заданию развёртывание в Yandex/VK Cloud либо Minikube пока не выполнено. Автоматическое создание тестовых ВМ и выполнение проверок дистрибутива также не входят в текущую итерацию. Эти границы учитываются при оценке результатов работы.

1 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

1.1 Микросервисная архитектура

Микросервисная архитектура предполагает разделение системы на самостоятельные приложения с явными сетевыми интерфейсами. Границы сервисов определяются предметной ответственностью: например, учёт версий ПО отделяется от управления инфраструктурными шаблонами и запусками. Каждое приложение может собираться, запускаться и масштабироваться независимо. Связь между сервисами задаётся контрактами API, а не прямым обращением к внутренним классам другого приложения.

Распределённость требует учитывать сетевые задержки, недоступность зависимостей и частичное выполнение операций. Поэтому для вызовов необходимы тайм-ауты и определённые реакции на ошибки. Обнаружение доступных экземпляров и декларативные HTTP-клиенты позволяют отделить бизнес-логику от управления сетевыми адресами. В Spring Cloud эти задачи поддерживаются, в частности, Eureka и OpenFeign [3, 4].

1.2 Централизованная конфигурация и Git

При наличии нескольких приложений повторение настроек в каждом модуле усложняет сопровождение. Централизованная конфигурация выделяет хранение и выдачу параметров в отдельный сервис. Spring Cloud Config поддерживает Git backend: настройки версионируются, а конкретный вариант определяется именем приложения, профилем и меткой Git [1].

Для локального прототипа допустим Git-репозиторий с URI file:. При этом Config Server работает с локальной рабочей копией. В проекте выбран label main; ответ сервера включает идентификатор коммита, что позволяет установить происхождение конфигурации. Пароли передаются через переменные окружения и не фиксируются в YAML-файлах. Получение конфигурации при старте следует отличать от её горячего обновления: последнее в данном прототипе не реализовано.

1.3 API Gateway, Discovery и балансировка

API Gateway задаёт единую точку входа: клиент обращается к согласованным путям API, а маршруты выбирают целевые сервисы. Spring Cloud Gateway Server WebFlux предоставляет инфраструктуру маршрутизации и фильтров [2]. В данной работе Gateway дополнительно проверяет токен доступа, а предметные сервисы повторяют проверку, чтобы доступ не зависел только от внешнего шлюза.

Service Discovery хранит сведения об экземплярах. Eureka-клиент регистрирует адрес и служебные метаданные, затем отправляет heartbeat-сообщения [3]. Это позволяет обращаться к логическому имени сервиса, даже если меняется число реплик. В маршрутах Gateway схема lb:// связывает имя сервиса с выбором экземпляра. Проверить балансировку можно серией запросов с фиксацией идентификатора ответившей реплики.

1.4 OAuth2 и межсервисные вызовы

OAuth 2.0 определяет механизм получения ограниченного доступа к защищённым ресурсам. В потоке client_credentials конфиденциальный клиент обращается к серверу авторизации от своего имени, а не представляет интерактивного пользователя [5]. Этот поток подходит для учебного API-клиента и демонстрационных автоматизированных запросов. Токен передаётся ресурсному сервису в заголовке Authorization: Bearer …; секрет клиента не передаётся предметным API.

В проекте токен представлен JWT, содержит издателя, аудиторию и scope. Права platform.read и platform.write разграничивают операции чтения и изменения. При межсервисном GET-вызове Feign передаёт входной токен, поэтому сервис-получатель применяет собственные правила доступа. OpenFeign позволяет описать вызов Java-интерфейсом с аннотациями Spring MVC и интегрируется с Eureka и Spring Cloud LoadBalancer [4].

1.5 Раздельное хранение и согласованность

Раздельные экземпляры БД позволяют предметным сервисам управлять собственной схемой и не зависеть от таблиц соседнего приложения. PostgreSQL используется как реляционная СУБД с транзакциями и ограничениями целостности [6]. В работе соответствие схемы и JPA-сущностей проверяется Hibernate, а первоначальное создание таблиц выполняется миграциями Flyway.

Ссылки на сущности других сервисов остаются идентификаторами в локальной записи. Перед созданием запуска они проверяются через API. Такая проверка не является общей транзакцией трёх баз: после неё состояние другой системы теоретически может измениться. В первой итерации изменение и удаление проектов и шаблонов через API отсутствуют. При дальнейшем развитии потребуются снимки параметров и явно определённые правила согласованности.

1.6 Связь платформы с ГОСТ Р 56939—2024

ГОСТ Р 56939—2024 имеет название «Защита информации. Разработка безопасного программного обеспечения. Общие требования». Он устанавливает требования к работам по созданию безопасного ПО и устранению недостатков, включая уязвимости [7]. Платформа испытаний может поддерживать учёт объекта проверки, условий выполнения и полученных результатов, но не заменяет весь процесс безопасной разработки.

Для ВКР предусмотрено развитие связи «требование → сценарий → сборка → стенд → результат». На текущем этапе сохраняются версия ПО, шаблон, имя набора тестов, время, итог и ссылка на артефакт. Точную применимость положений стандарта, состав проверок и доказательных материалов необходимо определить отдельно по полному тексту. Демонстрационная запись PASSED не означает сертификацию ПО или подтверждение соответствия всему стандарту.

2 ПРАКТИЧЕСКАЯ ЧАСТЬ

2.1 Архитектура и средства разработки

Предметная область выбрана в соответствии с темой выпускной квалификационной работы: «Инфраструктура-как-сервис: автоматизированная платформа тестирования классического клиент-серверного ПО по ГОСТ Р 56939—2024». Платформа предназначена для регистрации тестируемого ПО, описания тестовых сред и управления запусками проверок. Разработанный прототип реализует управляющее ядро этой платформы; создание виртуальных машин и автоматическое выполнение тестов являются следующими этапами.

Выделены три самостоятельных предметных микросервиса. `project-service` хранит название ПО, версию и ссылку на дистрибутив. `environment-service` хранит шаблоны тестовых стендов: ОС, образ, количество процессоров и объём памяти. `test-run-service` связывает проект, шаблон и набор проверок, фиксирует инициатора, состояние, время и результат запуска. Схема взаимодействия приведена на рисунке 1.

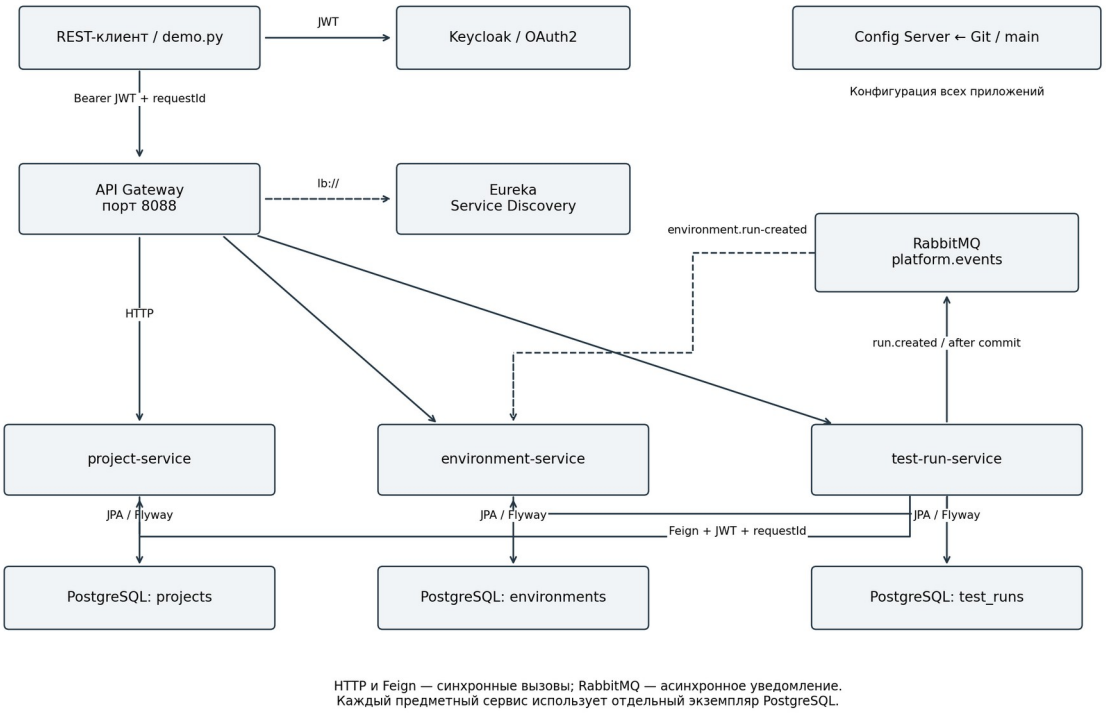


Рисунок 1 – Архитектура микросервисной платформы

Каждый сервис работает в отдельном процессе и собирается в самостоятельный исполняемый JAR. Пользовательские запросы проходят через Gateway. Для проверки ссылок сервис запусков обращается к реестру ПО и каталогу стендов через Feign, адреса экземпляров получает из Eureka. Между базами данных разных сервисов нет внешних ключей: согласование выполняется через API.

Таблица 1 – Компоненты системы

Компонент	**Назначение**
`project-service`	Регистрация и чтение версий тестируемого ПО; собственная БД projects
`environment-service`	Регистрация и чтение шаблонов стендов; собственная БД environments
`test-run-service`	Создание и сопровождение запусков, Feign-клиенты; собственная БД test_runs
`api-gateway`	Единая точка входа, проверка JWT, маршрутизация и балансировка
`discovery-server`	Eureka: регистрация экземпляров и обнаружение сервисов
`config-server`	Spring Cloud Config: выдача конфигурации из Git, ветка main
RabbitMQ	Обмен run.created: test-run-service → environment-service; аудит уведомления

Компонент	Назначение
Keycloak	OAuth2: выдача JWT сервисной учётной записи клиента platform-cli
Три PostgreSQL	Раздельные экземпляры БД с постоянными томами и миграциями Flyway

Использованы Java 21, Spring Boot 3.5.16, Spring Cloud 2025.0.3, Spring Data JPA, PostgreSQL 16.8, Flyway и Keycloak 26.2.5. Gateway использует WebFlux, предметные REST-сервисы — Spring MVC. Для модульных проверок применяются JUnit 5, Mockito и Spring Security Test. Развёртывание локального стенда описано файлами `Dockerfile` и `compose.yaml`.

2.2 Модель данных

Модель распределена между тремя предметными сервисами. Объект проекта идентифицирует версию ПО, шаблон описывает параметры будущей среды, а запуск связывает эти объекты с набором проверок. Основные поля приведены в таблице 2. Названия наборов и ссылки на артефакты в текущей версии хранятся как метаданные; автоматическая загрузка и исполнение дистрибутива не выполняются.

Таблица 2 – Основные сущности платформы

Сущность	Поля и назначение
Project	id, name, softwareVersion, artifactReference, createdAt — версия тестируемого ПО
EnvironmentTemplate	id, name, operatingSystem, imageReference, cpu, memoryMb — параметры шаблона стенда
TestRun	id, projectId, environmentId, testSuite, requestedBy, revision, status, createdAt, startedAt, finishedAt, summary, evidenceUri — состояние и результат запуска

Все первичные ключи имеют тип UUID. Внешние ссылки projectId и environmentId не являются межбазовыми внешними ключами и проверяются через сетевые API. Состояние запуска изменяется транзакционно внутри БД test-run-service. Изменение и удаление проектов и шаблонов в первой итерации через API не предусмотрены.

2.3 REST API и валидация запросов

REST API. Сервисы предоставляют операции регистрации проектов и шаблонов, создания запуска и сопровождения его жизненного цикла (таблица 3). Входные DTO проверяются Jakarta Bean Validation: обязательные строки не могут быть пустыми, для CPU и памяти заданы допустимые диапазоны. Идентификаторы сущностей представлены UUID.

Таблица 3 – Основные операции REST API

Метод и путь	Назначение
`POST /api/projects`	Регистрация версии ПО; HTTP 201
`GET /api/projects` `GET /api/projects/{id}`	Список проектов и получение одного проекта
`POST /api/environments`	Регистрация шаблона стенда; HTTP 201
`GET /api/environments` `GET /api/environments/{id}`	Список шаблонов и получение одного шаблона
`POST /api/test-runs`	Проверка внешних ссылок через Feign и создание запуска; HTTP 201
`GET /api/test-runs` `GET /api/test-runs/{id}`	Чтение списка и состояния запуска
`POST /api/test-runs/{id}/start`	Перевод QUEUED → RUNNING
`POST /api/test-runs/{id}/result`	Запись PASSED/FAILED, описания и ссылки на артефакт
`POST /api/test-runs/{id}/cancel`	Отмена QUEUED или RUNNING

2.4 Авторизация OAuth2 и права доступа

OAuth2. Keycloak выдаёт токен по потоку `client_credentials`. Клиент `platform-cli` использует сервисную учётную запись. Gateway и каждый предметный сервис проверяют JWT, издателя и аудиторию `platform-api`. GET-запросы требуют scope `platform.read`, операции записи — `platform.write`. Без токена возвращается 401, без необходимого scope — 403. Секрет клиента и пароли БД передаются через окружение и не включаются в отчёт.

Листинг 1 – Правила доступа предметных сервисов (SecurityConfig.java)

```
.authorizeHttpRequests(a -> a
    .dispatcherTypeMatchers(DispatcherType.ERROR).permitAll()
    .requestMatchers("/actuator/health", "/actuator/health/**").permitAll()
    .requestMatchers(HttpMethod.GET, "/api/**")
        .hasAuthority("SCOPE_platform.read")
    .requestMatchers("/api/**").hasAuthority("SCOPE_platform.write")
    .anyRequest().denyAll())
.oauth2ResourceServer(o -> o.jwt(j -> {}))
```

Для внутренних вызовов Feign передаёт входной Bearer-токен. Создание запуска требует прав записи и чтения, поскольку проверка проекта и шаблона выполняется GET-запросами к другим сервисам. Разрешение внутренней диспетчеризации `ERROR` сохраняет исходный HTTP-код ошибки: отсутствие сущности не подменяется ответом 403 при обращении контейнера сервлетов к обработчику ошибок.

2.5 Конфигурация из Git

Централизованная конфигурация. Исходные YAML находятся в `config-repo`. Скрипт `scripts/prepare-config.sh` копирует их в `.runtime/config-repo`, создаёт Git-репозиторий с веткой `main` и фиксирует изменения коммитом. Config Server использует Git backend с локальным URI `file:/config-repo`. Предметные приложения и Gateway импортируют настройки при запуске через `spring.config.import=configserver:…`. Для JGit рабочая копия подключается с правом записи, необходимым для служебного файла индекса.

Листинг 2 – Git backend Config Server (application.yaml, фрагмент)

```
spring:
  cloud:
    config:
      server:
        git:
          uri: ${CONFIG_GIT_URI:file:./.runtime/config-repo}
          default-label: main
          clone-on-start: true
```

2.6 Gateway и обнаружение сервисов

Gateway, Discovery и балансировка. Eureka хранит сведения о доступных экземплярах. В маршрутах Gateway используются имена

lb://project-service, lb://environment-service и lb://test-run-service. Spring Cloud LoadBalancer выбирает экземпляр сервиса. Для предметных сервисов не публикуются отдельные порты хоста, поэтому внешний клиент обращается к Gateway на http://localhost:8088.

Листинг 3 – Маршруты API Gateway (api-gateway.yaml, фрагмент)

```
routes:
  - id: projects
    uri: lb://project-service
    predicates:
      - Path=/api/projects,/api/projects/**
  - id: environments
    uri: lb://environment-service
    predicates:
      - Path=/api/environments,/api/environments/**
  - id: test-runs
    uri: lb://test-run-service
    predicates:
      - Path=/api/test-runs,/api/test-runs/**
```

2.7 Взаимодействие через FeignClient

Взаимодействие через Feign. Клиенты объявлены с именами сервисов без фиксированных URL (листинг 4). Перед сохранением запуска последовательно проверяются проект и шаблон. Отсутствующая ссылка даёт 422, недостаточные права зависимости — 403, её недоступность — 503. Для вызовов заданы тайм-ауты соединения 3 с и чтения 5 с. При неуспешной проверке запись запуска не создаётся. Общая распределённая транзакция между сервисами не используется.

Листинг 4 – Feign-клиенты проверки внешних ссылок

```
@FeignClient(name = "project-service")
public interface ProjectClient {
    @GetMapping("/api/projects/{id}")
    Reference get(@PathVariable("id") UUID id,
                  @RequestHeader("Authorization") String authorization);
    record Reference(UUID id) {}
}


@FeignClient(name = "environment-service")
public interface EnvironmentClient {
    @GetMapping("/api/environments/{id}")
    Reference get(@PathVariable("id") UUID id,
                  @RequestHeader("Authorization") String authorization);
    record Reference(UUID id) {}
}
```

2.8 Раздельные базы данных и миграции

Данные. Каждый предметный сервис использует собственный экземпляр PostgreSQL: `project-db`, `environment-db`, `test-run-db`. Схема создаётся миграцией `V1__*.sql`, после чего Hibernate проверяет соответствие сущностей схеме (`ddl-auto=validate`). Данные сохраняются в отдельных томах Docker. Реплики одного сервиса разделяют его БД; базы разных сервисов изолированы.

2.9 Управление состоянием запуска

Жизненный цикл запуска показан на рисунке 2. Новый запуск получает состояние `QUEUED`. Начать можно только ожидающий запуск, записать PASSED или FAILED — только выполняющийся. После завершения запрещены повторный старт и отмена. Поле `@Version` обеспечивает оптимистическую блокировку при параллельном изменении записи.

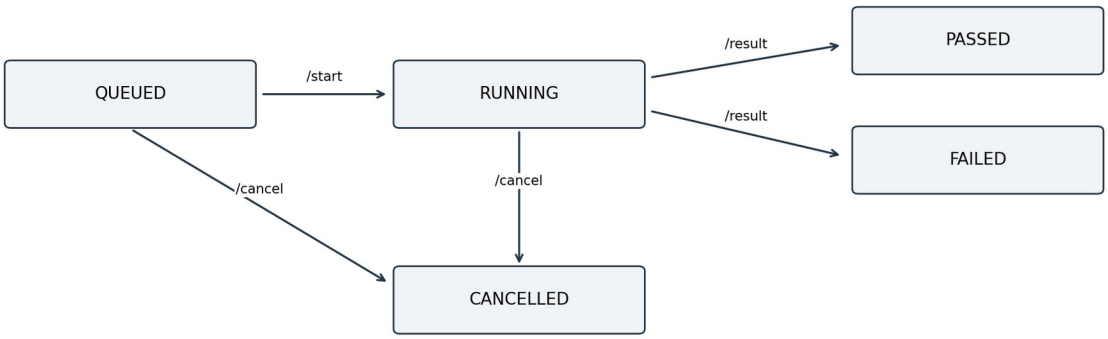


Рисунок 2 – Допустимые переходы состояния тестового запуска

На данном этапе методы `/start` и `/result` вызываются API-клиентом вручную. Статус PASSED в демонстрации означает успешную запись учебного результата, а не фактическое выполнение проверок дистрибутива. Ссылка `evidenceUri` сохраняется как метаданные; объектное хранилище и проверка существования файла пока не реализованы.

2.10 Модульное тестирование

Модульные тесты выполняются без запуска PostgreSQL и Keycloak: репозитории и Feign-клиенты заменяются Mockito-заглушками. Проверяются сохранение сущностей, ошибки поиска, передача токена и запрет создания запуска при неуспешной проверке ссылок. Отдельные тесты модели проверяют разрешённые и запрещённые переходы состояний.

MVC-тесты `ProjectApiTest` проверяют 401 без аутентификации, доступ с read scope, запрет записи с правами только на чтение, валидацию DTO и сохранение статуса 404 при внутренней диспетчеризации ошибки. JWT в этих тестах задаётся тестовым механизмом Spring Security; проверка реального токена Keycloak выполняется отдельно в сквозном сценарии.

Таблица 4 – Наборы модульных и MVC-тестов

Класс	Тестов	Проверяемое поведение
EnvironmentServiceTest	2	Сохранение шаблона; 404 при отсутствии записи
ProjectApiTest	5	Авторизация, scope, валидация и диспетчеризация ошибки
ProjectServiceTest	2	Сохранение проекта; 404 при отсутствии записи
RequestIdFeignConfigTest	2	Передача requestId в Feign; отсутствие заголовка вне HTTP
RunEventPublisherTest	2	Публикация после commit; отсутствие отправки при rollback
RunServiceTest	3	Проверка ссылок, передача токена; отсутствующая и недоступная зависимость
TestRunTest	5	Результат и время; недопустимые переходы; отмена запуска

Полная сборка проверена командой `./mvnw -B verify`. В текущей среде использован установленный OpenJDK 23.0.1 с целевой версией компиляции Java 21 и Maven 3.9.11. Все шесть Java-модулей собраны успешно; 21 тест завершился без ошибок и пропусков. Сводка получена из отчётов Surefire в каталогах `target/surefire-reports` (листинг 5).

Листинг 5 – Фактические результаты тестов (Surefire)

```
EnvironmentServiceTest: tests=2, failures=0, errors=0
ProjectApiTest: tests=5, failures=0, errors=0
ProjectServiceTest: tests=2, failures=0, errors=0
RequestIdFeignConfigTest: tests=2, failures=0, errors=0
RunEventPublisherTest: tests=2, failures=0, errors=0
RunServiceTest: tests=3, failures=0, errors=0
TestRunTest: tests=5, failures=0, errors=0
TOTAL: 21; failures=0; errors=0
```

2.11 Запуск и сквозная демонстрация

Локальный стенд запущен в Docker Desktop командой `./run.sh`. Скрипт подготавливает Git-репозиторий конфигурации и запускает Docker Compose. В обычной конфигурации работают одиннадцать контейнеров: шесть Java-приложений, Keycloak, RabbitMQ и три PostgreSQL. При проверке балансировки дополнительно запущен второй экземпляр project-service — всего двенадцать контейнеров. Состояние подтверждено командой `docker compose ps` (листинг 6).

Листинг 6 – Состояние контейнеров во время проверки балансировки

```
SERVICE                STATE      HEALTH     PORTS
api-gateway            running    -          127.0.0.1:8088->8080/tcp
config-server          running    healthy    127.0.0.1:8888->8888/tcp
discovery-server       running    healthy    127.0.0.1:8761->8761/tcp
environment-db         running    healthy    5432/tcp
environment-service    running    -          internal
keycloak               running    -          127.0.0.1:8180->8080/tcp
project-db             running    healthy    5432/tcp
project-service        running    -          internal
project-service-2      running    -          internal
rabbitmq               running    healthy    127.0.0.1:15672->15672/tcp
test-run-db            running    healthy    5432/tcp
test-run-service       running    -          internal
```

Config Server доступен на порту 8888. Проверка `/actuator/health` возвращает UP, а запрос `/project-service/default/main` — настройки из двух файлов и хеш Git-коммита. В листинге 7 приведены поля фактического ответа без значений параметров доступа.

Листинг 7 – Конфигурация из Git — поля фактического ответа

```
{
  "name": "project-service",
  "profiles": [
    "default"
  ],
  "label": "main",
  "version": "964a27e67e92feddedcb1a7301e9b5c10cf61985",
  "propertySources": [
    "file:/config-repo/project-service.yaml",
    "file:/config-repo/application.yaml"
  ]
}
```

Основной сценарий запускается командой `./scripts/demo.py`: клиент получает токен Keycloak, проверяет отказ без токена, создаёт версию ПО и шаблон Linux-стенда с 2 CPU и 2048 МБ RAM. Затем создаёт запуск, переводит его в RUNNING и сохраняет учебный результат PASSED. Повторный старт завершённого запуска возвращает 409, создание с отсутствующей ссылкой — 422. Результат выполнения приведён в листинге 8.

Листинг 8 – Результат сквозного демонстрационного сценария

```
Ожидание Keycloak и API (до 3 минут на компонент)...
401 без токена: OK
Проект, шаблон стенда, Feign-проверки и переходы запуска: OK
409 при повторном старте и 422 при отсутствующей ссылке: OK
Это демонстрация API. ВМ не создавались, тестирование дистрибутива не выполнялось.
```

После выполнения запись повторно прочитана из API по идентификатору ec3af512-3d29-4c60-8c77-b6fc909a82ab. Сохранены проект, шаблон, инициатор, время начала и окончания, версия записи, состояние, описание и ссылка на учебный артефакт. Это подтверждает согласованную работу Gateway, Keycloak, трёх предметных сервисов и отдельных баз данных; настоящее тестирование указанного дистрибутива демонстрационный клиент не выполняет.

Листинг 9 – Поля сохранённого тестового запуска

```
{
  "id": "ec3af512-3d29-4c60-8c77-b6fc909a82ab",
  "projectId": "c69e4bfd-18d1-472b-a9c8-9da7b4c42c3b",
  "environmentId": "20fb546c-b568-4ee1-8024-8834b92120c8",
  "testSuite": "manual-api-demonstration",
  "revision": 2,
  "status": "PASSED",
  "evidenceUri": "demo://manual-api-example"
}
```

Веб-страница Eureka открыта по адресу `http://localhost:8761`. В реестре присутствуют API-GATEWAY, ENVIRONMENT-SERVICE и TEST-RUN-SERVICE, а у PROJECT-SERVICE зарегистрированы два экземпляра со статусом UP. На рисунке 3 приведён снимок реально работающей страницы.

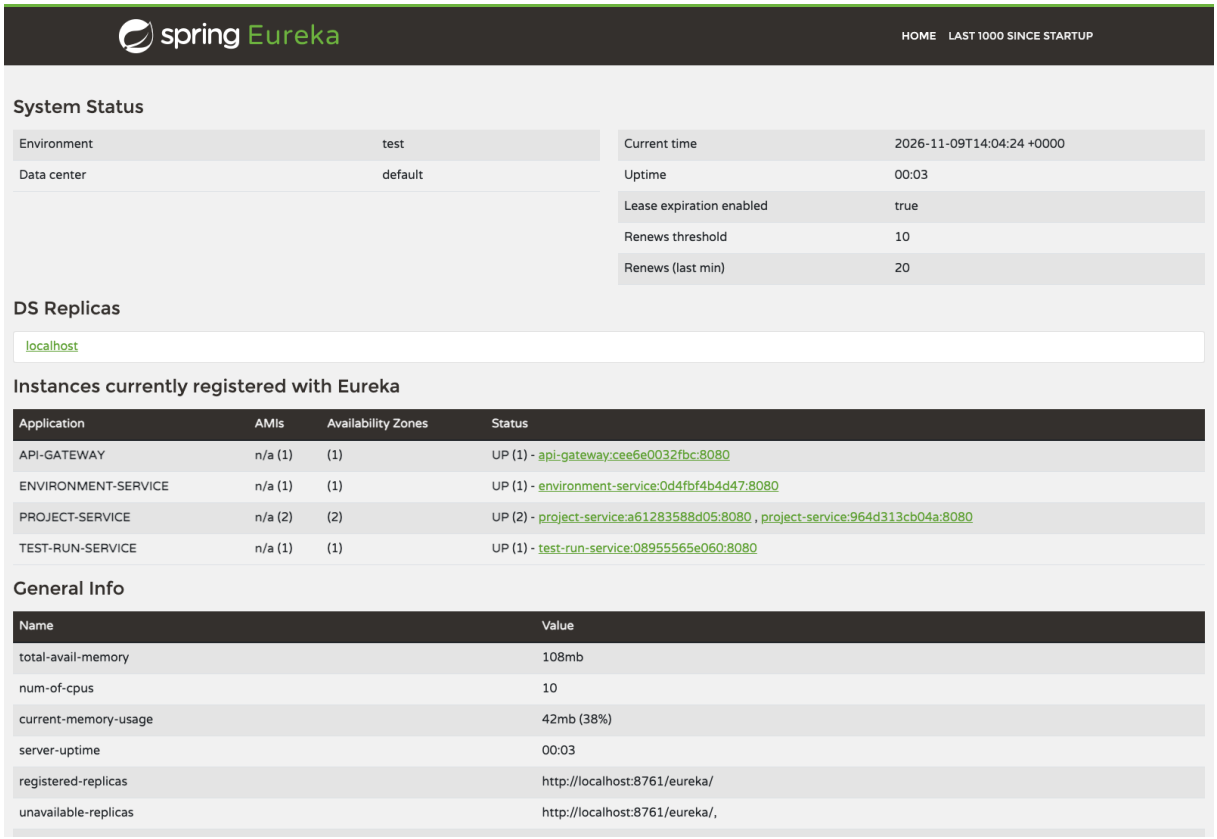


Рисунок 3 – Регистрация сервисов и двух экземпляров project-service в Eureka

2.12 Проверка балансировки нагрузки

Для проверки балансировки выполнены команды `docker compose up -d --scale project-service=2` и `./scripts/demo.py --balance`. Скрипт отправляет 12 GET-запросов через Gateway и учитывает заголовок `X-Service-Instance`. В фактическом результате оба экземпляра обработали по 6 запросов (листинг 10), что подтверждает распределение запросов между репликами.

Листинг 10 – Результат проверки балансировки

```
Ожидание Keycloak и API (до 3 минут на компонент)...
project-service:a61283588d05:8080: 6 ответов
project-service:964d313cb04a:8080: 6 ответов
```

Для трёх экземпляров PostgreSQL проверена история Flyway (листинг 11): в каждой базе успешно применена миграция версии 1. Обозначение `t` в выводе PostgreSQL означает значение true поля success.

Листинг 11 – Успешные миграции отдельных баз данных

```
project-db / projects: 1|projects|t
environment-db / environments: 1|environments|t
test-run-db / test_runs: 1|test runs|t
```

2.13 Пять этапов полного сценария

Для требований к материалам сдачи повторно выполнен сценарий на работающем Docker-стенде. Скрипт `scripts/verify_submission.py` сохраняет HTTP-ответы, идентификаторы запросов и статусы в `report/evidence/scenario.json`, исключая токены и пароли. По этим фактическим данным строятся локальные страницы протокола; снимки страниц сделаны Google Chrome в headless-режиме. Это снимки протокола проверки REST API, а не пользовательского интерфейса платформы. Первые три этапа показывают авторизацию, регистрацию ПО и шаблона стенда.

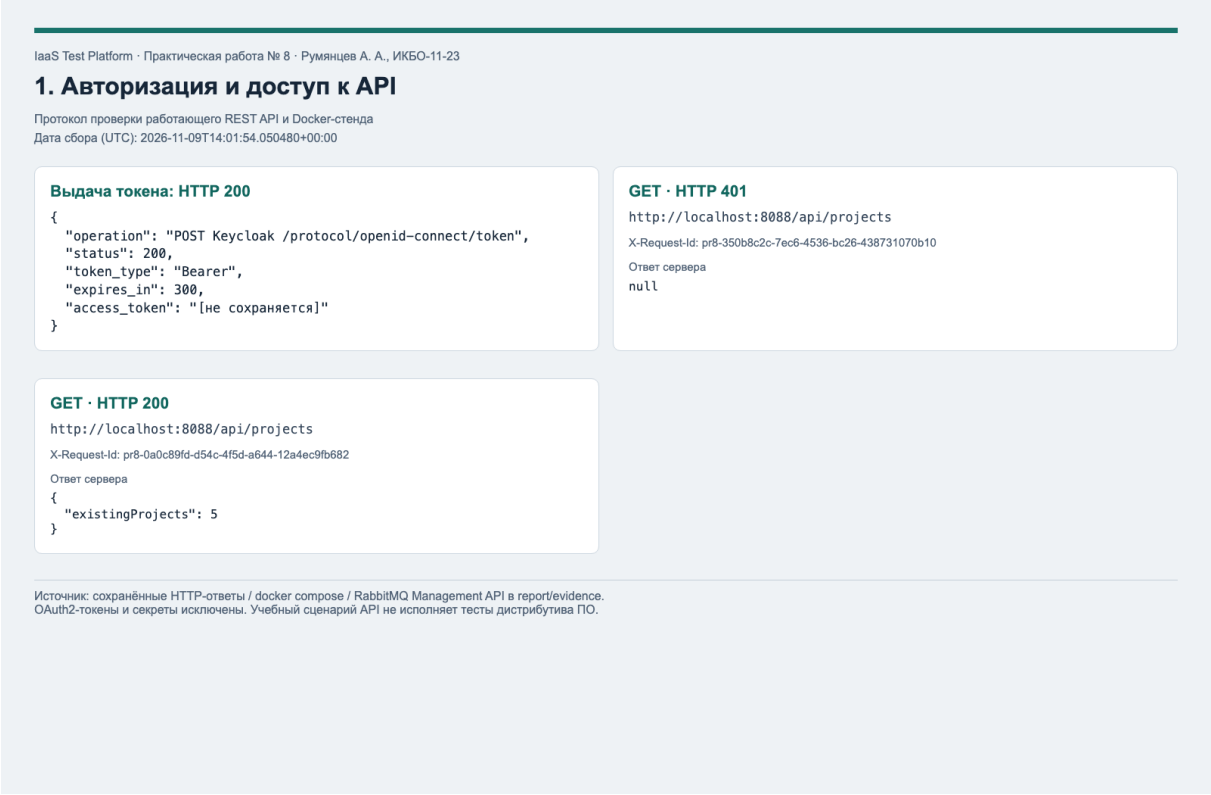


Рисунок 4 – Сценарий, этап 1: получение токена и проверка доступа

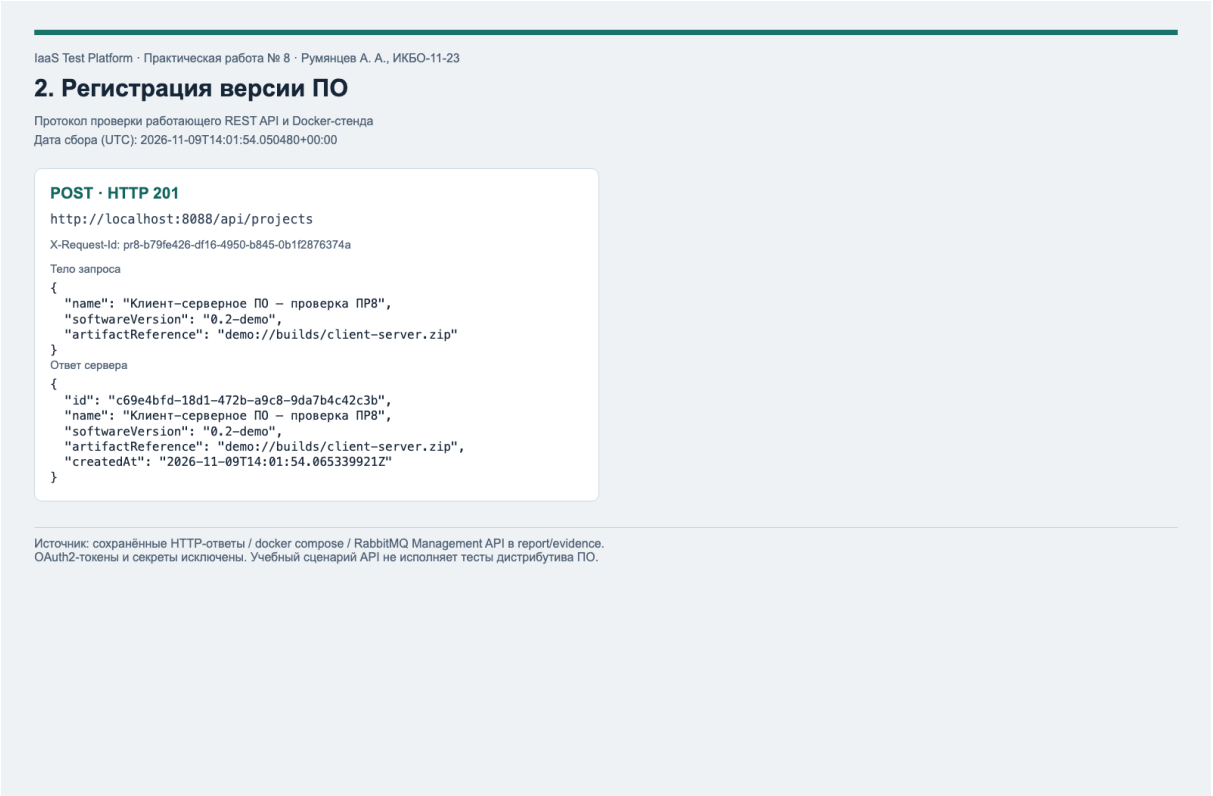


Рисунок 5 – Сценарий, этап 2: регистрация версии программного обеспечения

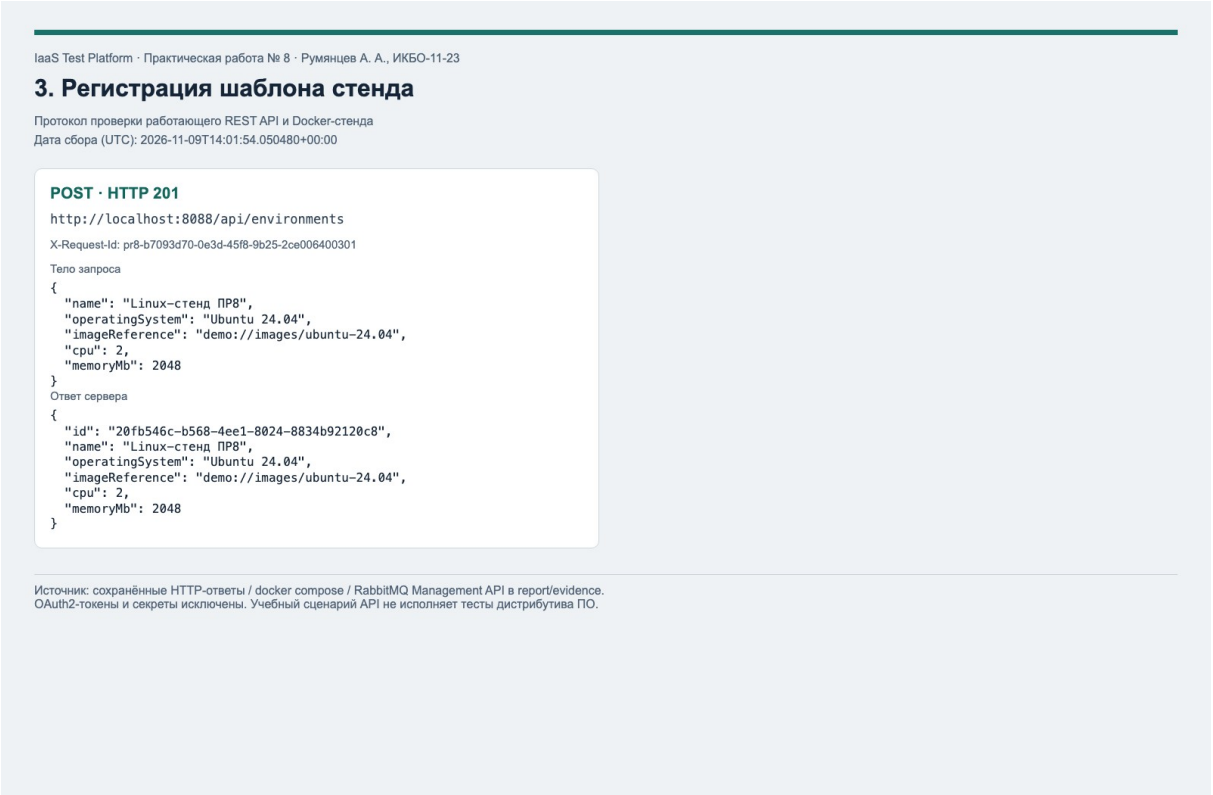


Рисунок 6 – Сценарий, этап 3: регистрация шаблона Linux-стенда

Четвёртый этап создаёт запись QUEUED только после проверки проекта и шаблона двумя Feign-вызовами. Пятый выполняет переходы

RUNNING и PASSED, повторно читает результат и проверяет HTTP 409 при повторном старте и HTTP 422 при отсутствующей ссылке. PASSED является учебной записью результата; запуск дистрибутива или тестовой виртуальной машины не выполнялся.

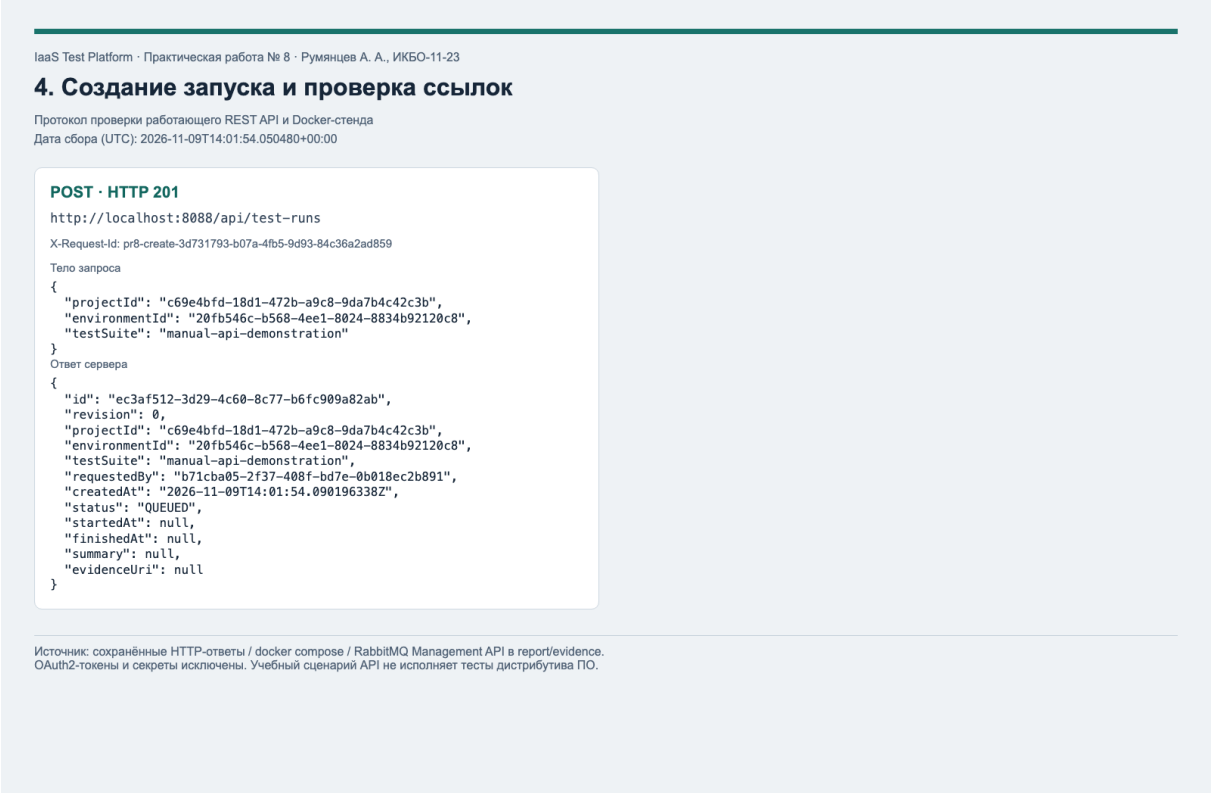


Рисунок 7 – Сценарий, этап 4: создание запуска после межсервисных проверок

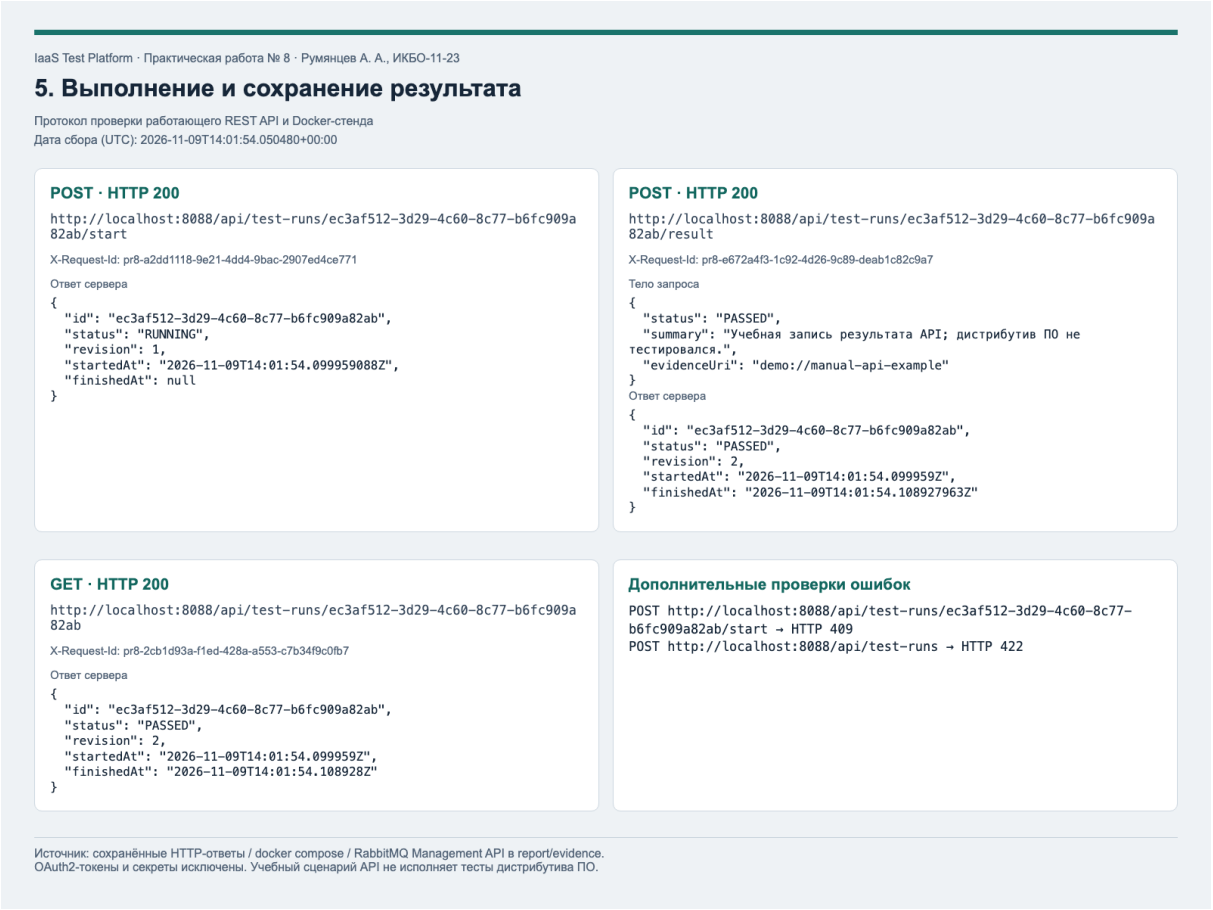


Рисунок 8 – Сценарий, этап 5: сохранение результата и обработка ошибок

Отдельно зафиксировано состояние работающих контейнеров. В момент сбора материалов запущены двенадцать контейнеров, включая RabbitMQ и две реплики project-service.

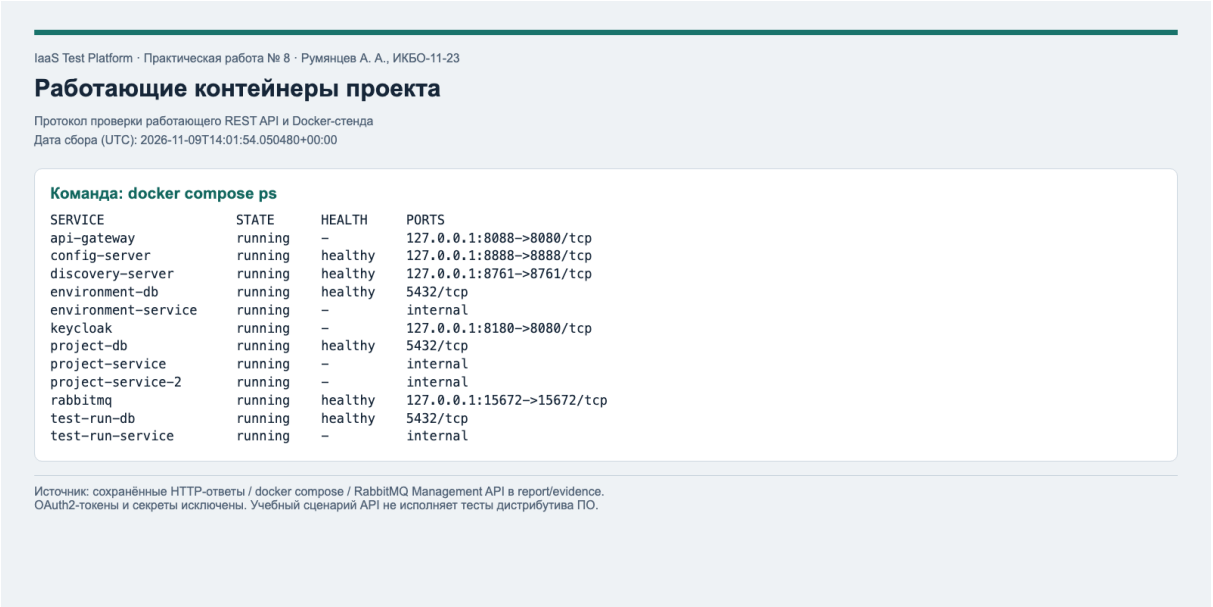


Рисунок 9 – Работающие контейнеры: Docker Compose, RabbitMQ и три PostgreSQL

2.14 Сквозной requestId через несколько сервисов

Gateway принимает заголовок `X-Request-Id` или создаёт UUID. Значение ограничено ста латинскими буквами, цифрами и символами точки, дефиса и подчёркивания. Заголовок передаётся предметному сервису, возвращается в ответе и фиксируется в журнале. Servlet-фильтры помещают его в MDC на время запроса и восстанавливают прежний контекст после завершения обработки. Feign RequestInterceptor переносит значение в оба зависимых GET-запроса.

Для одного `POST /api/test-runs` в журналах найдены строки Gateway, test-run-service, project-service и environment-service с одинаковым requestId. Внутренние GET-запросы завершились HTTP 200, внешний POST — HTTP 201. Исходный текст сохранён в `report/evidence/request-trace.txt`; на рисунке приведён снимок протокола этих строк.

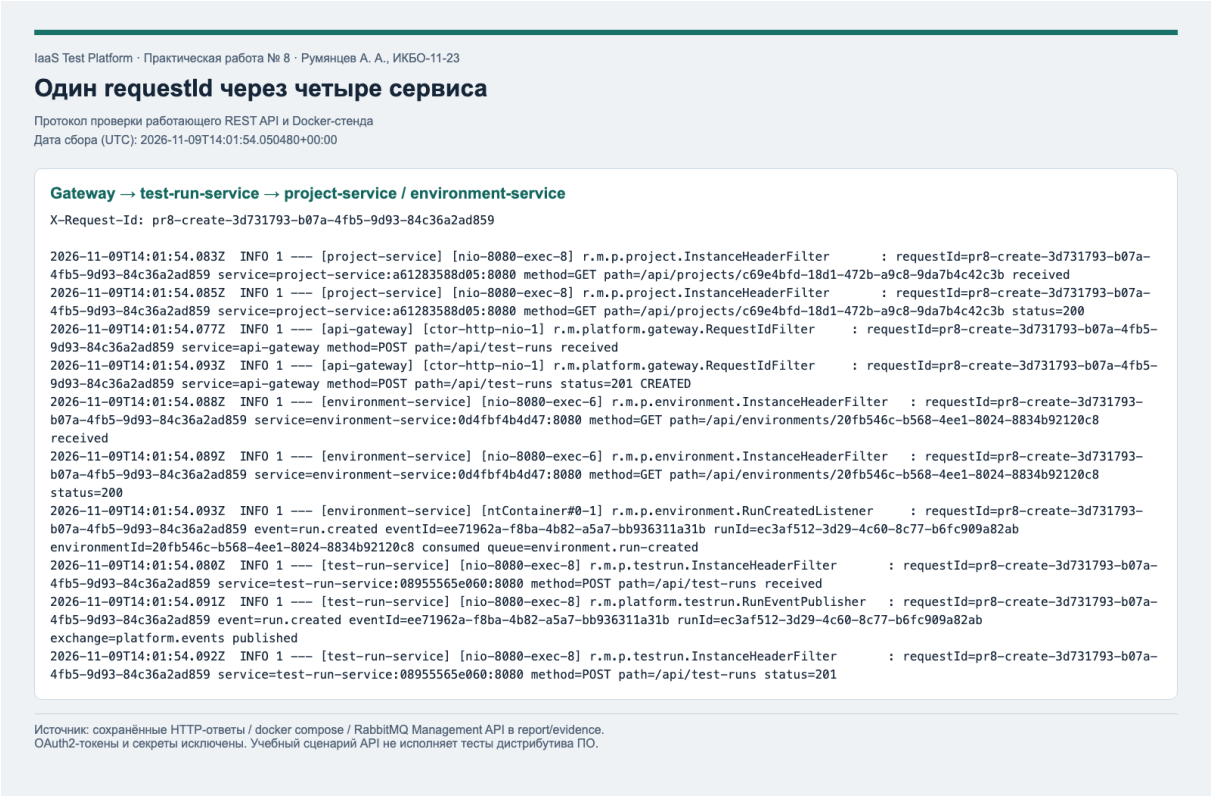


Рисунок 10 – Прохождение одного запроса через четыре сервиса по requestId

2.15 Асинхронное событие RabbitMQ

После успешного коммита создания запуска `@TransactionalEventListener` публикует JSON-событие `run.created` в durable exchange `platform.events`. Routing key `run.created` связан с durable queue `environment.run-created`. Сообщение содержит eventId, runId, environmentId и requestId. Отправитель — test-run-service, получатель — environment-service с `@RabbitListener`. Получатель фиксирует уведомление в журнале; выделение ВМ не выполняется.

На снимке протокола сопоставлены строки published и consumed с одним eventId и requestId. Статистика очереди дополнительно получена через RabbitMQ Management API. Тесты проверяют, что до коммита и при rollback отправка не происходит. Гарантированная повторная доставка после сбоя producer не реализована: между коммитом БД и отправкой остаётся окно потери события, для устранения которого потребуется transactional outbox. Ошибка отправки записывается в журнал без отката уже сохранённой сущности [8, 9].

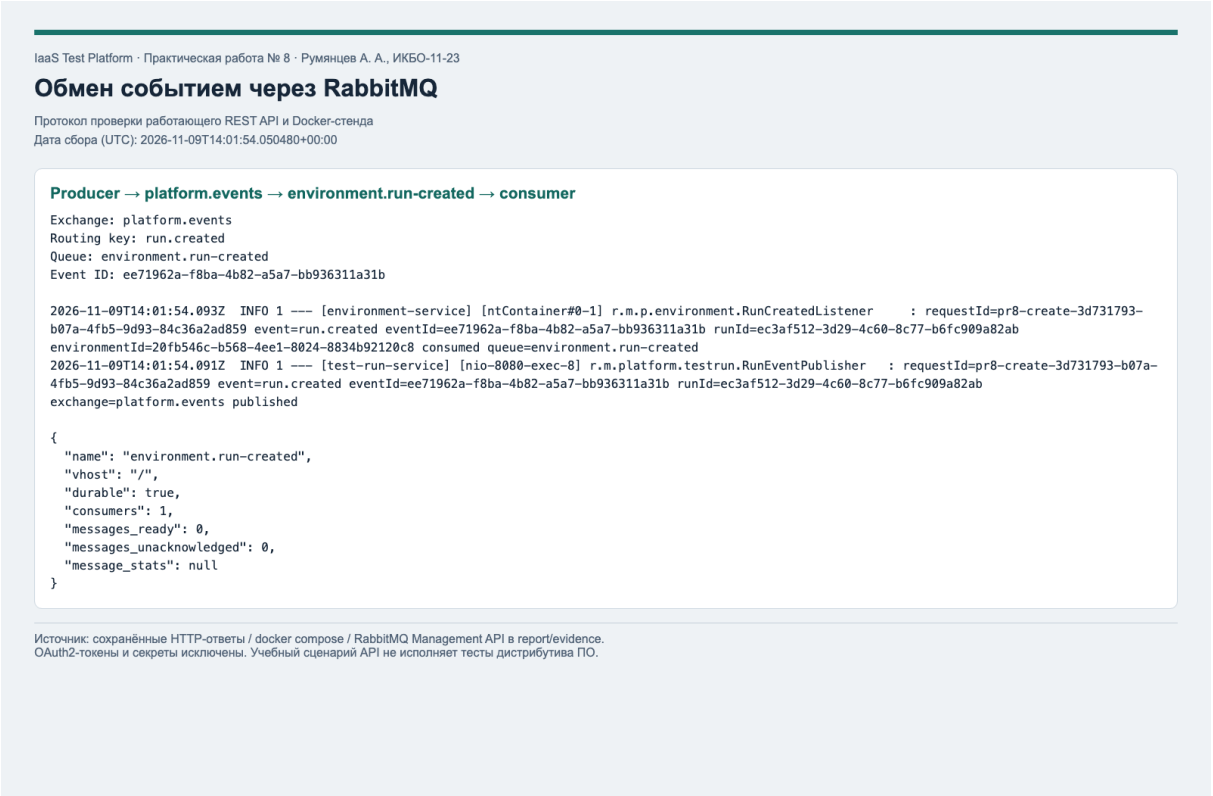


Рисунок 11 – Обмен run.created через RabbitMQ: совпадение eventId у producer и consumer

2.16 Отказ одного сервиса и восстановление

Для натурной проверки скрипт временно останавливает environment-service командой `docker compose stop environment-service`, затем пытается создать запуск через Gateway. Feign-вызов недоступной зависимости преобразуется в HTTP 503. Список идентификаторов запусков до и после неуспешного запроса совпадает: непроверенная запись не сохраняется. В это же время чтение проекта из project-service возвращает HTTP 200.

В блоке finally выполняется `docker compose start environment-service` с ожиданием готовности. После восстановления повторное создание запуска возвращает HTTP 201 и QUEUED. Таким образом, проверены частичный отказ, доступность независимого сервиса и возобновление сквозной операции. Фактические ответы и состояние контейнеров сохранены в `report/evidence/failure.json`.

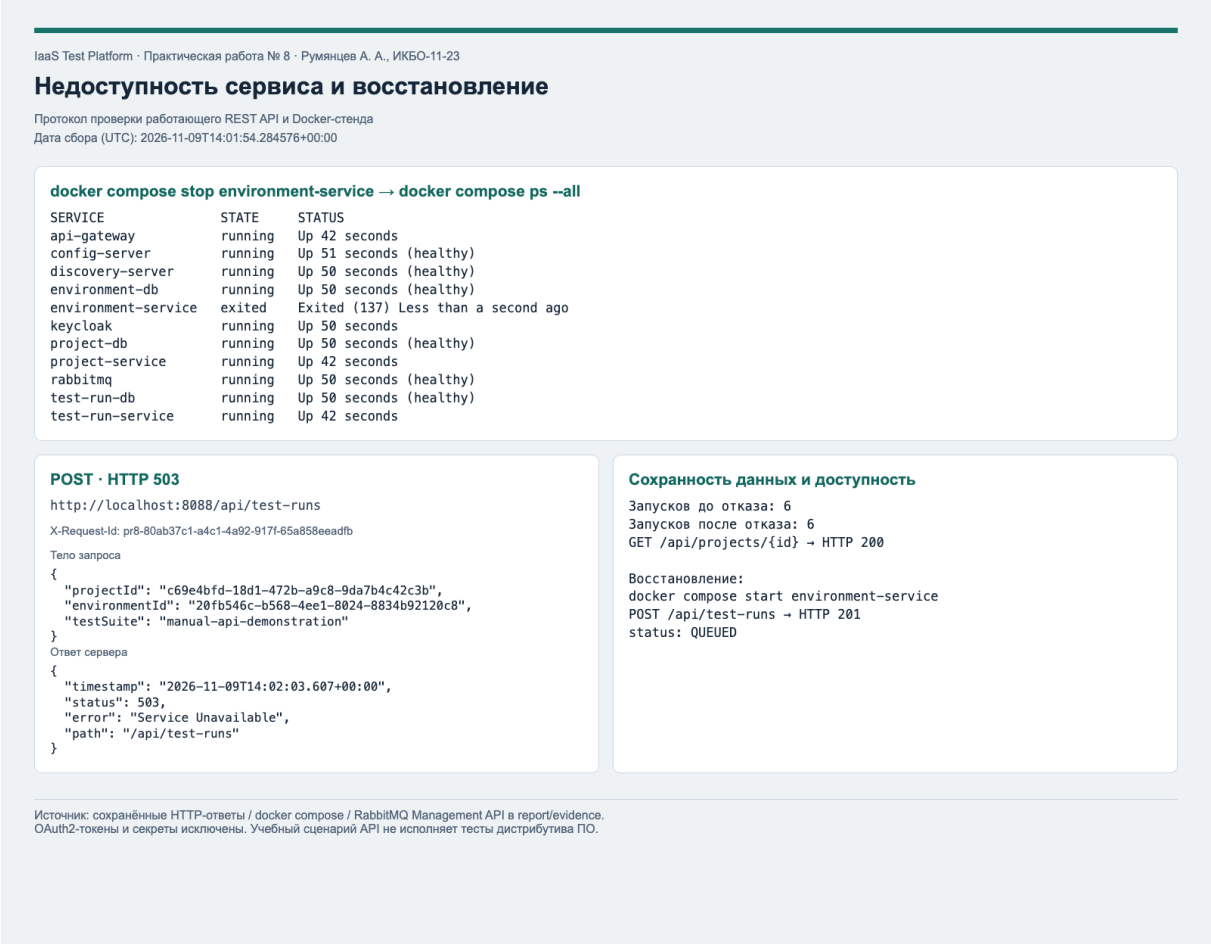


Рисунок 12 – Остановленный environment-service, HTTP 503 и восстановление до HTTP 201

2.17 Выполнение требований задания

В таблице 5 сопоставлены требования методических указаний и предоставленного перечня материалов с фактическими результатами. Проверки относятся к локальному Docker-стенду. Пункт о развёртывании в облаке или Minikube остаётся открытым и не считается выполненным по результатам запуска Docker Compose.

Таблица 5 – Проверка требований практической работы

Требование	Фактический результат
Авторизация OAuth2	Keycloak, client_credentials, JWT и scope; реальный токен проверен через API
Config Server с Git	Git backend, ветка main; подтверждены выдача настроек и хеш коммита
API Gateway	Единая точка входа на порту 8088; сквозной сценарий выполнен через неё
Service Discovery	Экземпляры предметных сервисов и Gateway зарегистрированы в Eureka

Требование	Фактический результат
Балансировка нагрузки	Две реплики project-service; 12 запросов распределены 6/6
Не менее 3 предметных сервисов	Реестр ПО, каталог шаблонов стендов, управление запусками
Взаимодействие FeignClient	Перед сохранением запуска проверяются проект и шаблон; ошибка ссылки даёт 422
Взаимодействие с БД	Spring Data JPA, PostgreSQL, миграции Flyway и сохранённые сущности
Отдельный экземпляр БД	Три контейнера PostgreSQL, отдельные базы и тома
Unit-тесты	21 тест; ошибок, неудач и пропусков нет
Файлы развёртывания	Dockerfile и compose.yaml; локальный контейнерный стенд запущен
Схема архитектуры в README	Mermaid-диаграмма, HTTP/Feign, БД, Config, Eureka и RabbitMQ
3–5 снимков полного сценария	Пять снимков протокола REST API: авторизация → объекты → запуск → результат
Снимок работающих контейнеров	12 контейнеров при двух репликах project-service
Сквозной requestId	Один идентификатор в Gateway, test-run-service и двух зависимых сервисах
Обмен Kafka/RabbitMQ	RabbitMQ: run.created, сопоставлены eventId публикации и потребления
Недоступность сервиса	Реальная остановка environment-service: 503 без записи; после старта 201
Репозиторий исходного кода	Ссылка вынесена в последний раздел отчёта
BootcampLabs	По указанию студента прикладывается отдельно
Yandex/VK Cloud либо Minikube	Пока не выполнено. Docker Compose не заменяет это требование задания

ЗАКЛЮЧЕНИЕ

Разработан и проверен локальный микросервисный прототип платформы тестирования клиент-серверного ПО: реестр проектов, каталог шаблонов стендов и управление тестовыми запусками. Реализованы OAuth2/JWT, Git Config Server, Gateway, Eureka, балансировка и Feign-вызовы с передачей токена. У каждого предметного сервиса отдельный экземпляр PostgreSQL. Полная Maven-сборка успешна, 21 тест пройден; сквозной сценарий подтвердил сохранение данных и обработку HTTP-ошибок. Проверка двух реплик показала распределение 12 запросов поровну. Один requestId прослежен через четыре сервиса; событие run.created доставлено через RabbitMQ. При реальной остановке environment-service получен 503 без лишней записи, после восстановления — 201.

Для выполнения оставшегося требования практической необходимо развернуть систему в Yandex/VK Cloud либо Minikube. Дальнейшее развитие ВКР включает создание реальных изолированных стендов, автоматическое исполнение проверок, хранение артефактов и прослеживаемость требований. ГОСТ Р 56939—2024 «Защита информации. Разработка безопасного программного обеспечения. Общие требования» относится к процессам безопасной разработки. Прототип поддерживает учёт объектов испытаний и результатов; соответствие всего процесса положениям стандарта по нему не заявляется.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Spring Cloud Config 4.3. Git Backend. Официальная документация [Электронный ресурс]. URL: https://docs.spring.io/spring-cloud-config/reference/4.3/server/environment-repository/git-backend.html (дата обращения: 09.11.2026).

2. Spring Cloud Gateway 4.3. Server WebFlux. Официальная документация [Электронный ресурс]. URL: https://docs.spring.io/spring-cloud-gateway/reference/4.3/spring-cloud-gateway-server-webflux.html (дата обращения: 09.11.2026).

3. Spring Cloud Netflix 4.3. Service Discovery: Eureka Clients. Официальная документация [Электронный ресурс]. URL: https://docs.spring.io/spring-cloud-netflix/reference/4.3/spring-cloud-netflix.html (дата обращения: 09.11.2026).

4. Spring Cloud OpenFeign 4.3. Declarative REST Client. Официальная документация [Электронный ресурс]. URL: https://docs.spring.io/spring-cloud-openfeign/reference/4.3/spring-cloud-openfeign.html (дата обращения: 09.11.2026).

5. Hardt D. The OAuth 2.0 Authorization Framework. RFC 6749. 2012 [Электронный ресурс]. URL: https://www.rfc-editor.org/rfc/rfc6749 (дата обращения: 09.11.2026).

6. The PostgreSQL Global Development Group. PostgreSQL 16 Documentation [Электронный ресурс]. URL: https://www.postgresql.org/docs/16/ (дата обращения: 09.11.2026).

7. Росстандарт. ГОСТ Р 56939—2024. Защита информации. Разработка безопасного программного обеспечения. Общие требования. Карточка стандарта [Электронный ресурс]. URL: https://protect.gost.ru/gost/details/f3818925-a96f-4f55-96e9-46b44720ee64 (дата обращения: 09.11.2026).

8. Spring AMQP. AmqpTemplate. Официальная документация [Электронный ресурс]. URL:

https://docs.spring.io/spring-amqp/reference/amqp/template.html (дата обращения: 09.11.2026).

9. Spring Framework. Transaction-bound Events. Официальная документация [Электронный ресурс]. URL: https://docs.spring.io/spring-framework/reference/data-access/transaction/event.html (дата обращения: 09.11.2026).

ПРИЛОЖЕНИЕ А

А.1 Структура проекта

Листинг 12 – Структура исходного кода и материалов

```
ПиРКСП_№8/
  pom.xml                    — родительский Maven-проект
  config-server/             — Git Config Server
  discovery-server/          — Eureka Server
  api-gateway/               — Spring Cloud Gateway
  project-service/           — реестр тестируемого ПО
  environment-service/       — шаблоны стендов
  test-run-service/          — запуски и Feign-клиенты
  config-repo/               — исходные YAML-настройки
  infra/keycloak/realm.json — OAuth2 realm и клиент
  scripts/prepare-config.sh — подготовка Git-репозитория
  scripts/demo.py            — демонстрация и балансировка
  scripts/verify_submission.py — requestId, RabbitMQ, отказ
  Dockerfile                 — сборка Java-приложений
  compose.yaml               — локальное развёртывание
  run.sh                     — запуск стенда
  docs/                      — проектная документация
  report/                    — отчёт, рисунки и результаты
```

А.2 Команды воспроизведения

Перед выполнением команд необходимо запустить Docker Desktop и открыть терминал в папке проекта. Для текущей среды в .env установлен GATEWAY_PORT=8088. Секреты берутся из локального файла окружения; приводить его содержимое в отчёте не требуется.

Листинг 13 – Запуск, проверка и остановка стенда

```
# Запуск
./run.sh
docker compose ps


# Проверка API
./scripts/demo.py


# Две реплики и балансировка
docker compose up -d --scale project-service=2
./scripts/demo.py --balance


# RequestId, RabbitMQ и отказ зависимости
python3 scripts/verify_submission.py --failure


# Сборка и тесты (JDK >= 21)
./mvnw -B verify


# Возврат к одной реплике
docker compose up -d --scale project-service=1


# Остановка с сохранением томов
docker compose down
```

Демонстрационный сценарий создаёт учебные записи. Повторный запуск добавляет новый проект, шаблон и запуск; он не проверяет дистрибутив и не создаёт ВМ. Для просмотра регистрации используется http://localhost:8761, для Keycloak — http://localhost:8180. Веб-интерфейс самой платформы в этой итерации не разрабатывался.

А.3 Репозиторий исходного кода

В репозитории размещены исходный код всех сервисов, Docker Compose, схема архитектуры в README, сценарии проверки, фактические материалы и итоговые DOCX/PDF. Локальный .env, токены, Maven-кеш и результаты сборки в репозиторий не включаются.

Ссылка на репозиторий проекта:

https://github.com/froid228/iaas-test-platform-pr8